



Amazon OpenSearch: From Search Engine to GenAI Vector Store

Search, Analytics, Shards, Serverless,
Semantic Search, Hybrid Search, and Vector
Retrieval

Presented by: Akash Patel
Professor: Victor Govindaswamy
Concordia University Chicago



Why Are We Learning OpenSearch?



In the previous weeks, I studied how data is prepared before it is used by Generative AI. Now the next question is how an application can search large amounts of prepared information quickly and return only the most useful results.



Amazon OpenSearch Service is important because it is built for search and analytics. It can search normal text and structured information, but it can also work with vector embeddings, which makes it useful in modern RAG and Generative AI applications.



This video starts with normal OpenSearch concepts before moving into vector search. I will first explain documents, indices, shards, replicas, storage, security, and serverless operation. After that, I will connect those ideas with semantic search, hybrid search, and Amazon Bedrock.



The main goal is not just to memorize OpenSearch terms. I want to understand **why these parts exist and how they work together when OpenSearch becomes the retrieval layer of an AI application.**

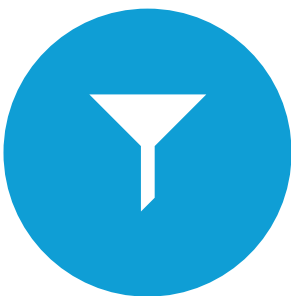
What Is Amazon OpenSearch?



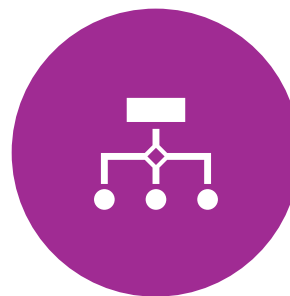
OpenSearch is a search and analytics platform. The course describes it as a search engine, analysis tool, visualization tool, and data-pipeline platform that can scale horizontally as the amount of data grows.



Search is only one part of its job. OpenSearch can also support full-text search, log analytics, application monitoring, security analytics, and clickstream analysis. This means the same general platform can be useful for many workloads where large amounts of information need to be searched or analyzed.



OpenSearch Dashboards provides a visual way to explore information. Instead of only sending API requests, users can work with dashboards and visualizations to understand what is happening inside their data.



Horizontal scaling means the workload can be spread across multiple nodes. Instead of depending on one very large machine, a cluster can distribute data and search activity across several machines as the workload becomes larger.

Documents, Indices, and Search Structure

-
- OpenSearch organizes information in a way that makes large-scale searching practical.
 - **A document is the individual piece of information we want to search.** It does not have to be only plain text. The course explains that structured JSON data can also be stored as a document, and each document has its own unique ID.
 - **An index is a searchable collection of documents.** Instead of searching every piece of information separately, OpenSearch organizes related documents inside an index so they can be searched together.
 - **The course also describes a type as the shared schema or mapping for similar documents.** For example, log records may share one structure, while encyclopedia-style articles may follow another structure.
 - **Simple Example**
 - Imagine an online store.
 - **Document:** One product
 - **Index:** All products
 - **Fields inside document:** Name, description, category, price, brand
 - If someone searches for “**wireless headphones**”, OpenSearch searches the product index and returns the documents that best match that request.

Why Does OpenSearch Use Shards?



An index can become too large to keep efficiently on only one machine. OpenSearch solves this problem by splitting an index into smaller pieces called **shards**.



Documents are assigned to particular shards. The course explains that documents are hashed to decide which shard will store them, and different shards can live on different nodes in the OpenSearch cluster.



Each shard is a complete Lucene index on its own. This allows different parts of the dataset to be stored and searched independently while still appearing to the application as one OpenSearch index.



The benefit is that **search and storage work can be distributed across the cluster**. If the dataset becomes larger, multiple nodes can share the workload instead of forcing one node to manage everything.



Easy Way to Think About It



A large book can be divided into several sections and given to different people to search at the same time. Each person searches their section, and the results are combined afterward.

Primary Shards and Replica Shards



OpenSearch also needs a way to protect data and continue serving searches when something goes wrong.



A primary shard is the main copy of a section of the index. When new information is written, the write request is first handled by the appropriate primary shard.



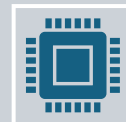
A replica shard is another copy of that primary shard. After the primary receives the update, the information is copied to its replica.



Read requests can be handled by either a primary or one of its replicas. This means replicas can help with availability and can also share some of the read workload.



If one node becomes unavailable, another node containing a replica can still hold a copy of that data. This is why replicas are important for building a more resilient search cluster.



The course example shows primary and replica shards distributed across several nodes rather than storing all copies together on one machine.

Amazon OpenSearch Service: Managed Option



Amazon OpenSearch Service is the AWS-managed version of OpenSearch. AWS handles much of the infrastructure work, while the user still chooses important settings such as capacity, storage, networking, and the overall domain configuration.



The managed service can be scaled up or down without downtime, but the course notes that this scaling is not automatically handled in the same way as the serverless option. The user still has responsibility for choosing an appropriate setup.



Costs can come from **instance hours, storage, and data transfer**, so architecture decisions still matter even though AWS manages the underlying service.



OpenSearch integrates with several AWS services. The course mentions connections with **Amazon S3, Kinesis Data Streams, DynamoDB Streams, CloudWatch, and CloudTrail.**



Zone Awareness can distribute OpenSearch resources across Availability Zones. This helps reduce the risk of one infrastructure failure affecting the entire search workload.

Hot, UltraWarm, and Cold Storage

- Not every piece of data needs the same level of performance.
- **Hot Storage**
- **Hot storage is used for active data that needs fast performance.** Standard data nodes use instance storage or EBS volumes so frequently searched and frequently updated indices can be accessed quickly.
- **UltraWarm Storage**
- **UltraWarm is intended for data that is searched but does not receive many new writes.** It uses Amazon S3 together with caching, which reduces cost compared with keeping everything on hot storage.
- **Cold Storage**
- **Cold storage is even lower cost and is designed for older information that is only needed occasionally.** The course gives examples such as periodic research or forensic analysis.
- **Main Idea**
- As information becomes older and less frequently used, it can move from:
- **Hot → UltraWarm → Cold**
- This helps balance **performance and cost** instead of keeping all historical data on the most expensive storage.

Index State Management

- Managing old indices manually would become difficult in a large system, so OpenSearch provides **Index State Management, or ISM**.
- **ISM allows rules to automatically manage an index as it becomes older.** Instead of an administrator remembering to change every index manually, a policy can decide what happens at different stages.
- For example, an index can automatically move from **hot storage to UltraWarm and later to cold storage** when the data no longer needs the fastest performance.
- A policy can also **change an index to read-only, reduce the number of replicas, create snapshots, or eventually delete old indices.**
- This is especially useful for workloads such as log data. A company may need yesterday's logs frequently, last month's logs occasionally, and very old logs only for audit or investigation.
- The main benefit is that OpenSearch can manage the **lifecycle of the data automatically instead of treating every index the same forever.**

Security in Amazon OpenSearch

- OpenSearch may contain business logs, internal documents, embeddings, or other sensitive information, so access control is important.
- **Resource-based policies can control who is allowed to access an OpenSearch resource.** This is useful for defining permissions directly around the domain or collection.
- **Identity-based policies use AWS IAM permissions.** These permissions can define what an AWS user or role is allowed to do with OpenSearch.
- **IP-based policies can restrict access according to network location.** This is useful when access should only come from trusted addresses or environments.
- OpenSearch can also use **request signing and VPC networking** so requests are authenticated and the service can be isolated inside a private AWS network.
- **Amazon Cognito can help secure user access to OpenSearch Dashboards.** This is useful when people need to log in and interact with dashboards instead of only calling the service through application APIs.

Performance Problems and OpenSearch Anti-Patterns

OpenSearch is powerful, but it is not the correct solution for every data problem.

Performance Issues

Too many shards can create JVM memory pressure. Each shard requires resources, so creating a very large number of small shards can make the cluster harder to manage instead of improving performance.

Unbalanced shard placement can also create problems. If one node receives much more work than others, that node may become overloaded even though the rest of the cluster still has available capacity.

Removing old or unused indices can help reduce unnecessary storage and memory pressure.

When OpenSearch Is Not the Right Tool

The course specifically says OpenSearch is **not designed for OLTP-style transactional workloads**. When an application needs transactions, RDS or DynamoDB may be a better fit.

For general ad-hoc querying of data, the course points toward **Athena** instead.

Main lesson: OpenSearch is mainly designed for **search and analytics**, so it should be selected because the workload actually needs those capabilities.

Amazon OpenSearch Serverless

- **OpenSearch Serverless removes much of the capacity planning required by a provisioned OpenSearch domain.** Instead of selecting and managing a fixed set of cluster instances, the service provides on-demand scaling.
- Serverless works with **collections rather than traditional provisioned domains**. These collections can be created for different workload types such as search or time-series data.
- **Encryption at rest is required**, and the service uses AWS KMS keys. Data-access policies and security policies control how users and applications can reach the collection.
- Capacity is measured using **OpenSearch Compute Units, or OCUs**. The user can still place an upper limit on capacity, so serverless does not mean the workload has no resource or cost considerations.
- The important difference is that the serverless model reduces the amount of infrastructure management while still providing OpenSearch search capabilities.

OpenSearch as a Vector Store

- This is where OpenSearch connects directly with Generative AI.
- **A normal search engine usually looks for matching words, but a vector store can search according to meaning.** Text, documents, or other information can first be converted into embeddings, and those embeddings can be stored inside OpenSearch.
- When a user asks a question, the question can also be converted into an embedding. OpenSearch then searches for stored vectors that are close to the query vector.
- **Amazon Bedrock Knowledge Bases can use OpenSearch as the vector-store layer.** This allows retrieved information from OpenSearch to become context for a Bedrock model.
- OpenSearch is not limited to Bedrock. The course also mentions integrations with **SageMaker AI, Hugging Face, and custom models** for RAG-style applications.
- **Simple Flow**
- **User Question → Create Query Embedding → Search OpenSearch → Retrieve Relevant Information → Send Context to Bedrock → Generate Answer**
- The model still generates the final response, but OpenSearch helps decide **which information the model should see first.**

Semantic Search vs. Hybrid Search

- **Semantic Search**
- **Semantic search uses vector similarity to search by meaning rather than only exact words.** Two sentences may use different words but still represent a similar idea, and their embeddings can help the system recognize that relationship.
- **Example:**
A user searches for “**How can I work from home?**”
The stored document says “**Remote work policy.**”
- A keyword-only search may depend heavily on matching words, but semantic search can recognize that these ideas are related.
- **Hybrid Search**
- **Hybrid search combines vector-based semantic search with traditional keyword search.** This can be useful when meaning matters but exact words, product codes, names, or metadata are also important.
- The course notes that hybrid search requires searchable keyword fields for metadata.
- **Why Both Can Matter**
- A Generative AI application may want the flexibility of semantic meaning while still keeping the precision of exact keyword or metadata matching.

How Vector Search Finds Similar Results



Searching every stored vector one by one would become slow when the vector database contains millions of items.



Exact Nearest Neighbor search compares very thoroughly, but the course notes that it can be slow. Because of this, large vector systems often use Approximate Nearest Neighbor, or ANN, methods.



OpenSearch supports vector-search engines such as **FAISS, NMSLib, and Apache Lucene.**



HNSW, or Hierarchical Navigable Small World, builds a graph that helps the system move quickly toward nearby vectors. It can provide high-quality search results, but it may require significant memory.



IVF, or Inverted File, groups vectors into areas before searching. This can work better with very large datasets because the system searches selected groups instead of examining every vector.



These methods involve trade-offs. Increasing search depth may improve recall, but it can also require more memory or make queries slower.



The important beginner idea is not to memorize every tuning value. It is to understand that ANN methods try to find **very similar results quickly without checking every possible vector.**

Example: OpenSearch in a Student Support GenAI System

- Imagine a university creates an AI assistant for students.
- **Step 1 – Store the Information**
- University policies, registration information, course guides, and student-service documents are prepared and converted into embeddings.
- **Step 2 – Store Embeddings in OpenSearch**
- OpenSearch keeps the vectors along with useful metadata such as:
 - Department
 - Document type
 - Academic year
 - Topic
 - Access level
- **Step 3 – Student Asks a Question**
- A student asks:
 - **“How many credits do I need to stay full-time?”**
- The question is converted into an embedding.
- **Step 4 – Retrieve Relevant Context**
- OpenSearch performs semantic or hybrid search and returns the most relevant sections from the university documents.
- **Step 5 – Bedrock Generates the Response**
- Those retrieved sections are added to the prompt sent to the Foundation Model.
- **Result:**
- The model answers using information that OpenSearch retrieved from the university's own data instead of relying only on what the model learned during training.
- This is the practical role of OpenSearch in many RAG applications: **search first, then generate.**

Key Takeaways & Thank You

- **OpenSearch is mainly a search and analytics platform.** Documents are organized into indices, and indices can be divided into shards so large workloads can be distributed across several nodes.
- **Replica shards add redundancy and can help handle reads**, while storage tiers and Index State Management help control the cost of older data.
- **Managed OpenSearch gives the user more control over the cluster, while OpenSearch Serverless reduces infrastructure management and provides on-demand scaling.**
- For Generative AI, **OpenSearch can store embeddings and perform semantic vector search.** Hybrid search can add keyword-based matching when exact terms and metadata also matter.
- ANN techniques such as **HNSW and IVF** allow large vector datasets to be searched efficiently without comparing every stored vector individually.
- The biggest idea I learned from this topic is that **OpenSearch can act as the retrieval engine between company data and a Foundation Model.** The Foundation Model generates the answer, while OpenSearch helps find the information that should support that answer.
- **Thank You**
- **Prepared by: Akash Patel**
- **Concordia University Chicago**
- **Special thanks to Professor Victor Govindaswamy for his continued guidance and support throughout this course. His explanations and course structure are helping me connect AWS services with real Generative AI use cases instead of only memorizing definitions. Learning how services such as OpenSearch connect with Amazon Bedrock has helped me understand how complete AI applications are designed on AWS.**